DO SITE AO PWA

Como preparamos o app para funcionar sem conexão?

Nove falas curtas explicam o que foi configurado, como testar e o que o modo offline realmente oferece.

DIVISÃO DAS FALAS

01 O que é PWA?
02 Plugin e ferramentas
03 Manifest
04 Ícones
05 Service worker
06 Cache do app
07 Cache das APIs
08 Build e teste offline
09 Resultado e limites

IMPORTANTE PARA A APRESENTAÇÃO

“Funcionar offline” significa abrir a estrutura do app e reutilizar conteúdo que já foi guardado. A primeira visita exige internet; respostas e imagens nunca carregadas podem não estar disponíveis sem conexão.

A ideia: o que é um PWA?

FALA SUGERIDA

“Nosso projeto já era um site feito com React e Vite. A proposta foi acrescentar recursos para ele se comportar um pouco mais como um aplicativo: poder ser instalado pelo navegador e abrir mesmo quando a conexão falhar. PWA significa Aplicativo Web Progressivo. Não foi necessário reconstruir o site do zero: adicionamos uma configuração que prepara o app para instalação e cache. O conteúdo continua sendo o mesmo: perfil, personagem de Dragon Ball, Pokémon e filme.”

O QUE MOSTRAR OU DESTACAR

- A tela atual do app.
- A diferença entre abrir um site e instalar um PWA.

TERMO EM LINGUAGEM SIMPLES

PWA = site com recursos de aplicativo.

PASSAGEM PARA A PRÓXIMA PESSOA

“Primeiro, preparamos as ferramentas do projeto.”

Preparação: plugin e ferramentas

FALA SUGERIDA

“O app usa Vite para desenvolver e montar a versão de produção. Instalamos o pacote vite-plugin-pwa como dependência de desenvolvimento. Esse plugin conecta o Vite aos recursos PWA e gera arquivos importantes durante o build. A configuração foi feita na pasta interna Vite-project, onde estão o package.json e o vite.config.js. Essa distinção importa porque existe uma pasta externa com nome parecido.”

O QUE MOSTRAR OU DESTACAR

- package.json: vite-plugin-pwa e comandos do projeto.
- vite.config.js: integração do plugin com Vite.

TERMO EM LINGUAGEM SIMPLES

Plugin = ferramenta que acrescenta recursos ao projeto.

PASSAGEM PARA A PRÓXIMA PESSOA

“Com as ferramentas prontas, demos uma identidade ao app.”

Identidade: o manifest

FALA SUGERIDA

“O manifest é como a ficha de identidade do aplicativo. Nele configuramos o nome Meu Universo Geek, o nome curto, a descrição, o idioma português, a cor do tema e o modo standalone. Standalone faz com que, quando instalado, o app possa abrir em uma janela própria, sem a barra normal do navegador. O navegador lê essa ficha para saber como apresentar o app ao usuário.”

O QUE MOSTRAR OU DESTACAR

- vite.config.js: nome, idioma, cores e display.
- dist/manifest.webmanifest: arquivo gerado no build.

TERMO EM LINGUAGEM SIMPLES

Manifest = ficha com os dados de apresentação do PWA.

PASSAGEM PARA A PRÓXIMA PESSOA

“Também criamos imagens para o ícone aparecer bem em diferentes telas.”

Ícones: reconhecimento visual

FALA SUGERIDA

“Um aplicativo instalável precisa de ícones apropriados. O projeto gera imagens PNG de 192 por 192 e 512 por 512 pixels, além de uma versão maskable, que pode ser recortada por alguns dispositivos. Também gera um ícone para a tela inicial da Apple. O script icons:generate cria esses arquivos a partir do desenho do projeto, então é possível refazê-los sem desenhar cada tamanho à mão.”

O QUE MOSTRAR OU DESTACAR

- public/: ícones PNG e ícone vetorial.
- package.json: comando npm run icons:generate.

TERMO EM LINGUAGEM SIMPLES

Maskable = ícone preparado para diferentes formatos de recorte.

PASSAGEM PARA A PRÓXIMA PESSOA

“Agora veremos quem ajuda o app a lembrar seus arquivos.”

Service worker: o ajudante do navegador

FALA SUGERIDA

“O service worker é um pequeno programa que o navegador executa separado da página. Ele pode interceptar pedidos de arquivos e procurar uma cópia guardada. O plugin gera esse programa e registra a atualização automática. No build, aparecem arquivos como sw.js e registerSW.js. Assim, quando uma nova versão do app é publicada, o service worker pode atualizar a cópia usada pelo navegador.”

O QUE MOSTRAR OU DESTACAR

- dist/sw.js: service worker gerado.
- dist/registerSW.js: código de registro.

TERMO EM LINGUAGEM SIMPLES

Service worker = ajudante que intermedeia pedidos e cache.

PASSAGEM PARA A PRÓXIMA PESSOA

“O ajudante guarda primeiro os arquivos que formam o próprio site.”

Cache do app: a estrutura offline

FALA SUGERIDA

“No build, configuramos o Workbox para guardar a estrutura publicada: HTML, JavaScript, CSS, ícones e algumas imagens locais. Essa lista é chamada precache. É como deixar uma cópia da casca do app pronta para abrir sem internet depois que a pessoa visita o site. O comando de produção mostrou quantos arquivos entraram no precache. Isso não significa que todas as respostas externas e todas as imagens da internet também foram guardadas.”

O QUE MOSTRAR OU DESTACAR

- vite.config.js: globPatterns do precache.
- Terminal: quantidade de arquivos incluídos no precache.

TERMO EM LINGUAGEM SIMPLES

Precache = guardar arquivos do app antes de precisar deles offline.

PASSAGEM PARA A PRÓXIMA PESSOA

“Além dos arquivos, o app também consulta APIs.”

APIs: guardar respostas recentes

FALA SUGERIDA

“O site busca informações em APIs de perfil, Dragon Ball, Pokémon e filmes. Configuramos uma regra NetworkFirst para esses endereços. Isso quer dizer: primeiro tenta buscar uma resposta nova pela internet; se a rede falhar, pode usar uma resposta guardada no cache. O cache tem limite de quantidade e validade de até sete dias. Só é possível reutilizar respostas que tenham sido carregadas com sucesso antes.”

O QUE MOSTRAR OU DESTACAR

- vite.config.js: hosts das APIs e regra NetworkFirst.
- Cache: até 80 respostas, com validade de sete dias.

TERMO EM LINGUAGEM SIMPLES

NetworkFirst = tenta internet primeiro; usa cache se necessário.

PASSAGEM PARA A PRÓXIMA PESSOA

“Uma informação pessoal já fica guardada diretamente no aparelho.”

Build e ensaio do modo offline

FALA SUGERIDA

“Para testar o PWA, não basta abrir o servidor de desenvolvimento. Rodamos npm run build, que cria a versão final na pasta dist, e depois npm run preview, que serve essa versão localmente. Primeiro abrimos o site com internet para o navegador registrar o service worker e guardar arquivos e respostas. Depois podemos usar as ferramentas do navegador, na aba Network, marcar Offline e recarregar. Também podemos procurar a opção de instalar no menu do Chrome ou Edge.”

O QUE MOSTRAR OU DESTACAR

- Terminal na pasta Vite-project: npm run build e npm run preview.
- Chrome/Edge: Network !’ Offline; instalação pelo menu do navegador.

TERMO EM LINGUAGEM SIMPLES

Build = preparar os arquivos finais para publicação.

PASSAGEM PARA A PRÓXIMA PESSOA

“Por fim, vamos explicar o que funciona offline e os limites do teste.”

Resultado, limites e próximos passos

FALA SUGERIDA

“O resultado é um site que pode ser instalado e cuja estrutura pode abrir offline depois de uma primeira visita online. Respostas de APIs consultadas podem ser reaproveitadas enquanto estiverem no cache. A primeira visita ainda precisa de internet; dados nunca carregados, novas evoluções e imagens hospedadas fora do app talvez não apareçam offline. Para publicar para outras pessoas, precisamos hospedar o site em HTTPS. Nosso próximo passo é testar em diferentes dispositivos e, se necessário, guardar também os dados e imagens importantes no próprio aparelho.”

O QUE MOSTRAR OU DESTACAR

- Demonstração: visite online, depois teste Offline.
- Explique com clareza o que já tem cache e o que depende da rede.

TERMO EM LINGUAGEM SIMPLES

Offline não é magia: só funciona com conteúdo previamente guardado.

PASSAGEM PARA A PRÓXIMA PESSOA

“Obrigada(o)! Essa foi a transformação do nosso site em PWA.”